UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+ ACCREDITED UNIVERSITY

Mental Health Text Checker Using Sentiment Analysis

Student Name: Anuj **Section/Group: 24MAM-2A**
UID: 24MCI10020 **Subject Code: 24CAP-774**
Branch: MCA(AIML) **Date of Performance: 07-04-2026**
Subject Name: AI TOOLS Lab **Semester: 4th**

Github: https://github.com/anujdhiman28/Mental-Health-Text-Checker-Using-Sentiment-Analysis.git

TABLE OF CONTENTS

1. Abstract
2. Introduction
3. Objective
4. Methodology
5. Hardware and Software Requirements
6. System Implementation
7. Output Screenshot
8. Future Scope
9. Conclusion

1. Abstract

This mini project report presents the design and implementation of a desktop application titled "Mental Health Text Checker" that uses artificial intelligence techniques to perform basic sentiment analysis on user-entered text. The main aim of the application is to provide the user with a quick indication of whether the written text appears to contain positive or negative emotional content and to display simple, friendly suggestions when negative sentiment is detected.

The system is developed in Python using the built-in Tkinter library for the graphical user interface and the Transformers library for accessing a pre-trained Natural Language Processing model that performs sentiment analysis. By combining these technologies, the project shows how modern NLP models can be integrated into small, easy-to-use tools that run on a personal computer.

The application accepts user input through a multi-line text box, processes the text using a sentiment-analysis pipeline, and then displays the predicted label and confidence score along with a colored status message and a short suggestion. Although the system does not attempt to replace professional mental health support, it can help users reflect on how their written communication may appear to others and encourage them to maintain a healthier mindset when expressing their feelings in text form.

2. Introduction

In the modern digital world, people communicate extensively through text messages, emails, social media posts, and online forums. Many emotions, including happiness, frustration, sadness, and anxiety, are expressed through written language. However, users may not always be aware of how negative or harsh their own words sound until they receive some form of feedback on the emotional tone of their text.

Natural Language Processing (NLP) is a branch of artificial intelligence that focuses on enabling computers to understand, interpret, and generate human language. Sentiment analysis is a common NLP task in which text is automatically classified as positive, negative, or neutral based on its emotional content. Such techniques are widely used in industry to analyze customer reviews, comments, and opinions shared on the internet.

For this mini project, sentiment analysis is used in a small, user-facing application rather than on large datasets. The goal is to give individual users a

simple tool with which they can experiment by typing sentences and immediately seeing how an AI model classifies their text. This helps them to become more aware of the emotional impression created by their words.

Tkinter is the standard Python interface to the Tk GUI toolkit and is available with most Python installations. It provides a variety of widgets such as labels, buttons, frames, and text boxes that can be combined to build cross-platform desktop applications without additional installation effort.In this project Tkinter is used to design a simple window for entering text and viewing results.

The Hugging Face Transformers library simplifies the use of state-of-the-art NLP models by providing ready-made pipeline APIs for common tasks such as sentiment analysis, text generation, and question answering. Instead of training models from scratch, developers can load a pre-trained model and run it with only a few lines of code. This makes it ideal for students and beginners who want to focus on application logic rather than model training.

This introduction provides the background needed to understand the remaining sections of the report, which explain the objectives, methodology, implementation details, and possible future improvements for the "Mental Health Text Checker" application.

3. Objective

The project is guided by a set of clear educational, technical, and user-focused objectives:

- To design and develop a simple desktop application that analyzes the sentiment of user-entered text related to thoughts and feelings using Natural Language Processing techniques.
- To provide an intuitive graphical user interface using Python Tkinter so that even non-technical users can easily interact with the sentiment checker and obtain results.
- To integrate a pre-trained sentiment analysis model through the Hugging Face Transformers pipeline("sentiment-analysis"), demonstrating practical use of modern NLP tools in a mini project.
- To classify text mainly into positive and negative categories and display the predicted label along with its confidence score inside the GUI.
- To show gentle, supportive suggestions when negative or stressful text is detected, encouraging users to relax, talk to someone, or take a short break.

- To create an extendable mini project that can later be improved with multi-emotion detection, multiple language support, and additional features while remaining simple to understand for students.

4. Methodology

The methodology adopted for the "Mental Health Text Checker" project can be described as a sequence of well-defined steps:

Problem definition and scope – The first step is to define the project as a basic sentiment based "Mental Health Text Checker" that gives feedback on emotional tone, not a clinical diagnostic tool. The scope is limited to identifying whether the text appears broadly positive or negative so that the system remains manageable within the boundaries of a mini project.

Selection of tools and libraries – Python is chosen as the programming language, Tkinter as the GUI framework, and Hugging Face Transformers as the NLP library to load a pre-trained sentiment-analysis model via the pipeline API. This combination reduces development effort and allows the focus to remain on integration and user experience.

Design of GUI layout – A single main window is designed containing a title label at the top, one multi-line text box for input, a larger multi-line text box for output, and two buttons for "Check Mental State" and "Clear". Appropriate background and text colors are selected so that the interface is visually clear and easy to understand for users.

Model initialization and integration – The Transformers library is imported and the sentiment-analysis pipeline is created using pipeline("sentiment-analysis"). This call loads a suitable pre-trained model. The resulting classifier object is stored in a variable and reused every time the user requests analysis, avoiding repeated loading of the model.

Input handling and validation – When the user clicks the "Check Mental State" button, text is read from the Tkinter input Text widget and stripped of leading or trailing spaces. If the input is empty, the system inserts a warning message into the output area asking the user to enter text instead of calling the model with an empty string.

Sentiment analysis process – For valid input, the stripped text is passed to the classifier pipeline. The pipeline returns a dictionary containing a sentiment label such as POSITIVE or NEGATIVE and a numerical confidence score that expresses how strongly the model supports its prediction. This output forms the basis for decision making in the next step.

Decision logic and message generation – A simple if–else block checks the returned label. If the label is NEGATIVE, the application assumes the text may express stress or negative emotion and sets a status message such as "Possible Stress / Negative Emotion" along with a supportive suggestion. Otherwise, it treats the text as positive or normal and displays a green status line encouraging the user to maintain a healthy mindset.

Result display in GUI – The program then writes into the output Text widget a header line such as "----- Analysis Result -----", followed by the sentiment label and score, the status line, and the suggestion. Tkinter text tags are used so that titles appear in blue, negative messages in red, and positive messages in green, enabling quick visual interpretation of the result.

Clear and reset functionality – A separate clear() function is implemented to delete the contents of both the input and output text boxes. The "Clear" button is linked to this function, allowing users to reset the interface and test multiple sentences one after another.

Testing and refinement – Finally, the application is tested manually with various positive, negative, and mixed sentences to verify that predictions appear correctly, empty inputs show warnings, colors are applied properly, and the GUI remains responsive. Observations from this testing phase are used to make minor adjustments and improvements to the interface and messages.

5. Hardware and Software Requirements

The hardware and software requirements for running the "Mental Health Text Checker" application are modest and suitable for a typical personal computer or laptop:

Hardware Requirements:

- A personal computer or laptop with a modern 64-bit processor (for example, Intel or AMD).
- At least 4 GB of RAM to comfortably run Python, the GUI, and the sentiment-analysis model.
- Approximately 1–2 GB of free disk space for installing Python, libraries, and cached model files.
- A display with a resolution of 1280x720 or higher to view the complete Tkinter window.

Software Requirements:

- Operating System: Windows, Linux, or macOS capable of running Python 3.
- Python 3.x installed on the system.
- Tkinter library (included with most standard Python distributions) for the desktop GUI.
- Transformers library from Hugging Face to provide the sentiment-analysis pipeline.
- A deep learning backend such as PyTorch or TensorFlow required by the selected Transformers model.
- Optional: A code editor or IDE such as VS Code, PyCharm, or IDLE for easier development and debugging.

6. System Implementation

The implementation of the "Mental Health Text Checker" is carried out entirely in Python. The program starts by importing the Tkinter module using "import tkinter as tk" and importing the pipeline function from the Transformers library. The sentiment-analysis pipeline is created by calling pipeline("sentiment-analysis"), which loads a suitable pre-trained model for general sentiment tasks. This classifier object is stored in a global variable and reused every time the user runs a new analysis.

The main Tkinter window is created using tk.Tk(). Its title is set to "Mental Health Text Checker" and an appropriate geometry (width and height) is specified. The background color is set to black to provide a high contrast with the white input area and light yellow output area. A title label is added at the top using a bold font, giving the application a clear and friendly heading.

Below the title label, a multi-line Text widget is created for user input. This widget allows the user to enter multiple lines of text, such as sentences or short paragraphs describing their feelings or general thoughts. Another multi-line Text widget is placed below the input area to display the results of the sentiment analysis. The size of this output widget is chosen so that it can show several analyses if the user runs the tool multiple times.

Tkinter text tags are configured to control the appearance of specific parts of the output. A tag named "title" is defined to display the analysis header in blue. Two additional tags, "red" and "green", are defined to color negative status messages in red and positive status messages in green. These tags are applied whenever the program inserts lines marked as titles or status messages.

The check_mental_health() function is responsible for handling the main button click. It uses the get() method of the input Text widget to read the user text from the first character to the end, then strips whitespace. If no text is entered, the function inserts a warning message into the output box and returns. If text is present, the function calls the classifier pipeline and retrieves the first element of the returned list, which contains the sentiment label and score.

Based on the label, the function chooses a status message and suggestion as described in the methodology section. Then it inserts into the output box a header line, the label and score, the status line with the appropriate color tag, and the suggestion. This design allows the user to see both the raw model output and a human-readable interpretation.

The clear() function provides a simple way to reset the interface. It uses the delete() method on both the input and output Text widgets to remove all content. A "Check Mental State" button is connected to the check_mental_health() function and a "Clear" button is connected to the clear() function. Both buttons are given background colors (green for check and red for clear) to make their roles immediately obvious.

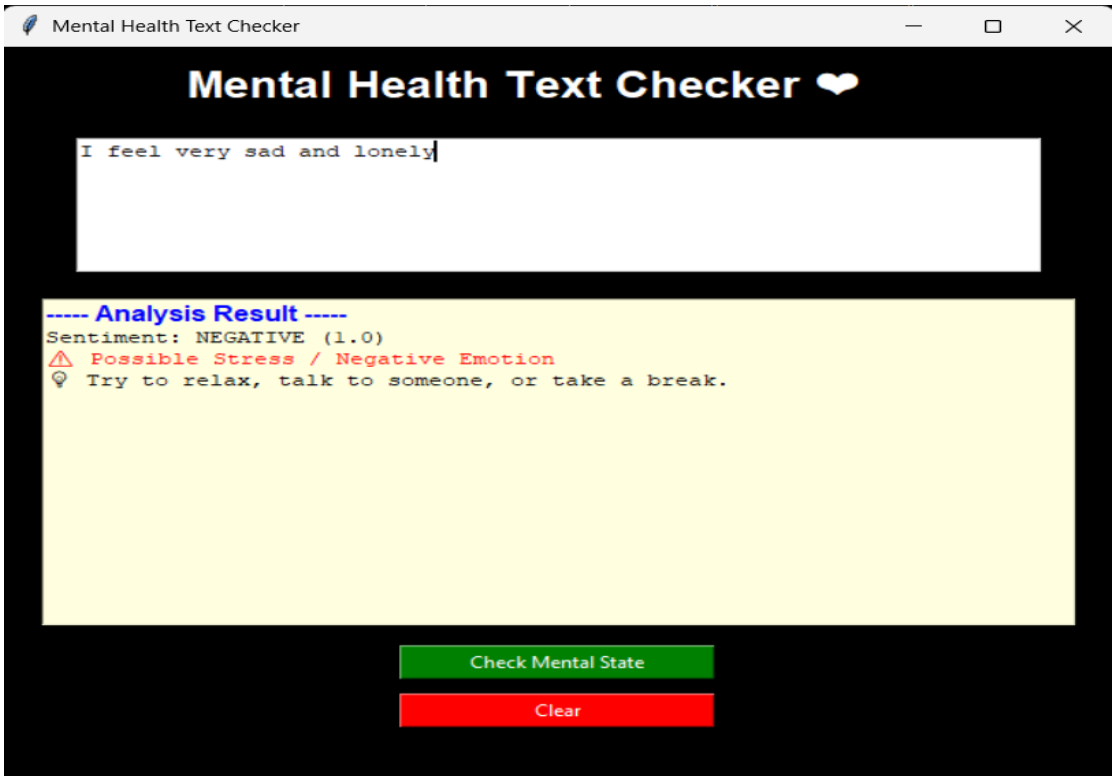
Finally, root.mainloop() is called to start Tkinter's event loop. From this point, the application waits for user actions such as typing, clicking buttons, or closing the window. Each button click triggers the appropriate callback function, allowing the user to repeatedly analyze new pieces of text without restarting the program.

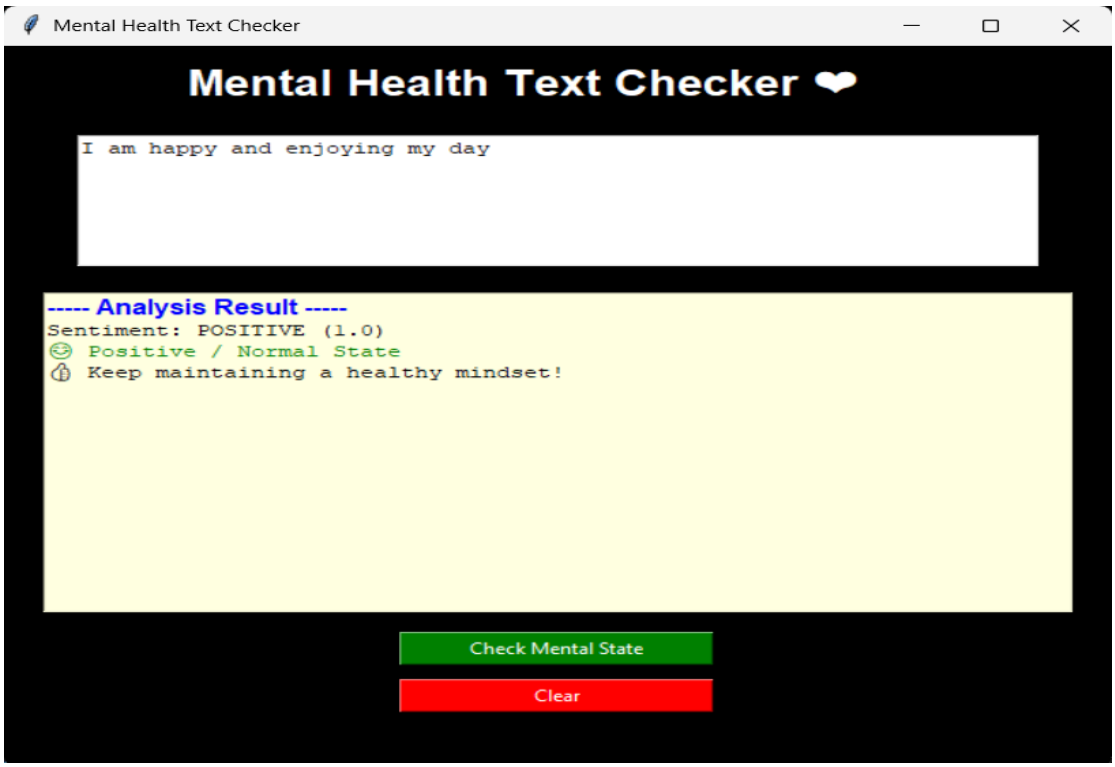
7. Output Screenshot

When the application is executed, the main window appears with the title label at the top, the input text box, the output text box, and the two buttons at the bottom. After the user types a sentence, for example "I am feeling very tired and sad today", and clicks on the "Check Mental State" button, the output area is updated with the analysis result.

The header line "----- Analysis Result -----" is displayed in blue. Below this, a line shows the predicted sentiment label (for instance, NEGATIVE) and the confidence score returned by the model. If the label is negative, a red-colored line reports "Possible Stress / Negative Emotion" and a brief suggestion encourages the user to relax or talk to someone. For a clearly positive sentence, the status line appears in green and indicates a positive or normal emotional state.

In the printed report, a screenshot of this window should be inserted at this point to give a clear visual representation of the final output. The screenshot should capture the entire window, including the title, the text boxes, a sample input text, and the displayed analysis result.





8. Future Scope

Although the current version of the "Mental Health Text Checker" provides a basic demonstration of sentiment analysis in a GUI environment, there are many opportunities for future enhancement. One interesting direction is to move beyond binary sentiment into multi-class emotion detection. More advanced models can classify text into emotions such as joy, sadness, anger, fear, and surprise, which would allow the application to give more detailed feedback.

Another possible improvement is to support multiple languages. Many users feel more comfortable expressing their emotions in their native language, and multilingual models are increasingly available. Adding a language selection feature and loading different models would significantly increase the usefulness and inclusiveness of the tool.

Additional GUI features could also be added. For example, the application could store a history of analyzed texts, provide export options, or allow the user to customize colors and fonts. Integration with messaging platforms, email clients, or chatbots could enable real-time feedback while users are typing messages.

From an educational perspective, the project could be extended to include explanatory panels that describe how the underlying NLP model works, including tokenization, embeddings, and attention mechanisms. This would make the tool valuable not only as a sentiment checker but also as a teaching aid for students learning about AI and NLP.

Finally, there is scope to incorporate stronger ethical guidelines. Future versions of the application should emphasize that it is not a substitute for professional mental health services and could include links to helplines or counseling resources when highly negative text is detected. Such features would help to align the project with responsible AI practices.

9. Conclusion

The "Mental Health Text Checker" mini project successfully demonstrates how graphical user interface programming can be combined with modern Natural Language Processing tools to create a useful and educational application. Using Python, Tkinter, and the Transformers sentiment-analysis pipeline, the system provides real-time feedback on the emotional tone of user-entered text.

By working on this project, students gain practical experience in multiple areas: using external libraries, designing GUIs, handling user input and output, and

understanding the limitations of automatic sentiment analysis. They also confront important questions about the appropriate use of AI in sensitive domains such as mental health.

Although the application is not intended to diagnose or treat mental health conditions, it can still play a small role in encouraging self-reflection and awareness about language use. When combined with clear disclaimers and resources for seeking professional help, such tools can contribute positively to digital well-being.

In summary, the project meets its defined objectives, runs on commonly available hardware and software, and offers many opportunities for further enhancement. The concepts and techniques used here can be extended to more advanced projects involving sentiment analysis, emotion detection, chatbots, and intelligent user interfaces.